

MODULE 15

Business Logic and Service Layer Design

Building the core of your application — organize business logic into a clean, testable, maintainable service layer.





MODULE 15 OVERVIEW

Building the core of your application — the Service Layer contains business logic, orchestrates operations, and manages transactions.

- The Service Layer sits between controllers and the data access layer — it enforces rules, manages transactions, and acts as the bridge between presentation and persistence.

DURATION & LAB



1 Hour Theory

Service/Repository patterns, DI, transactions.



45 Minutes Lab

Service Layer Design.



Scenario

Enforce customer status-transition rules for Northstar CRM.

SERVICE LAYER DOES

Business rules, orchestration, transactions, exceptions

SITS BETWEEN

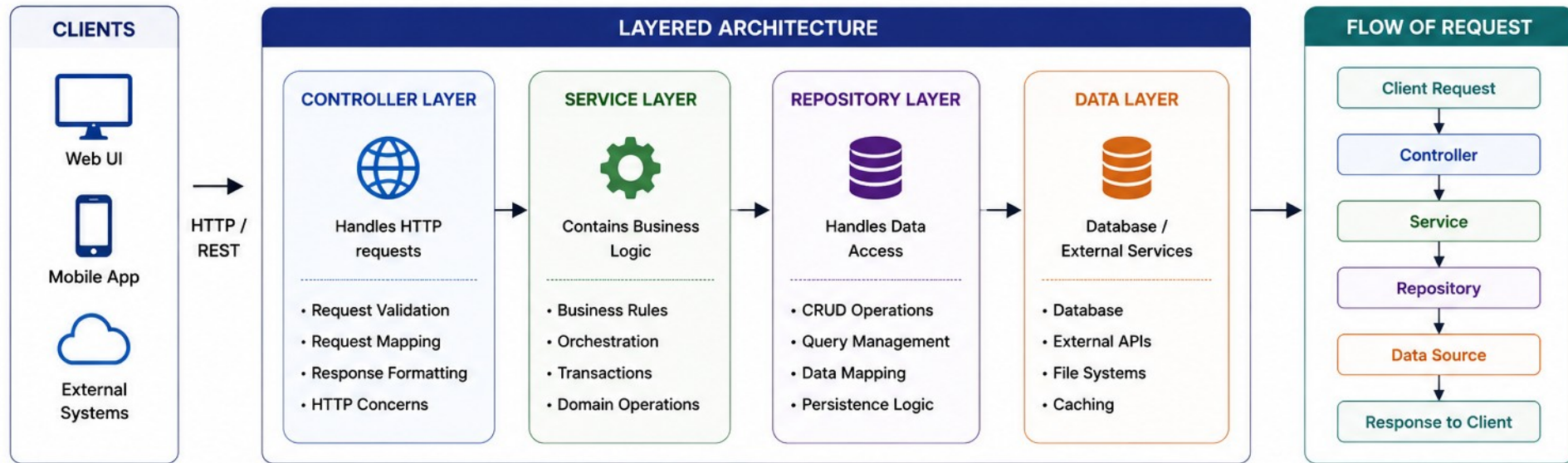
Controllers (presentation) and Repositories (data access)

THE PAYOFF

Clean separation, testability, reusability, scalability

KEY TAKEAWAY A well-designed service layer keeps your business logic organized, reusable, and independent of web and data-access concerns.

SERVICE LAYERS MATTER



SEPARATION OF CONCERNS

Each layer has one job — controllers handle HTTP, services handle business logic, repositories handle data access.

Presentation Layer Controllers handle HTTP requests and responses.

Service Layer Business logic, orchestration, transactions.

Repository Layer Data access — CRUD operations via JPA/DAO.

Database Persistent storage.

WITHOUT: logic scattered across controllers, duplicated, hard to unit test.

WITH: logic centralized, reusable, and easy to test independently.

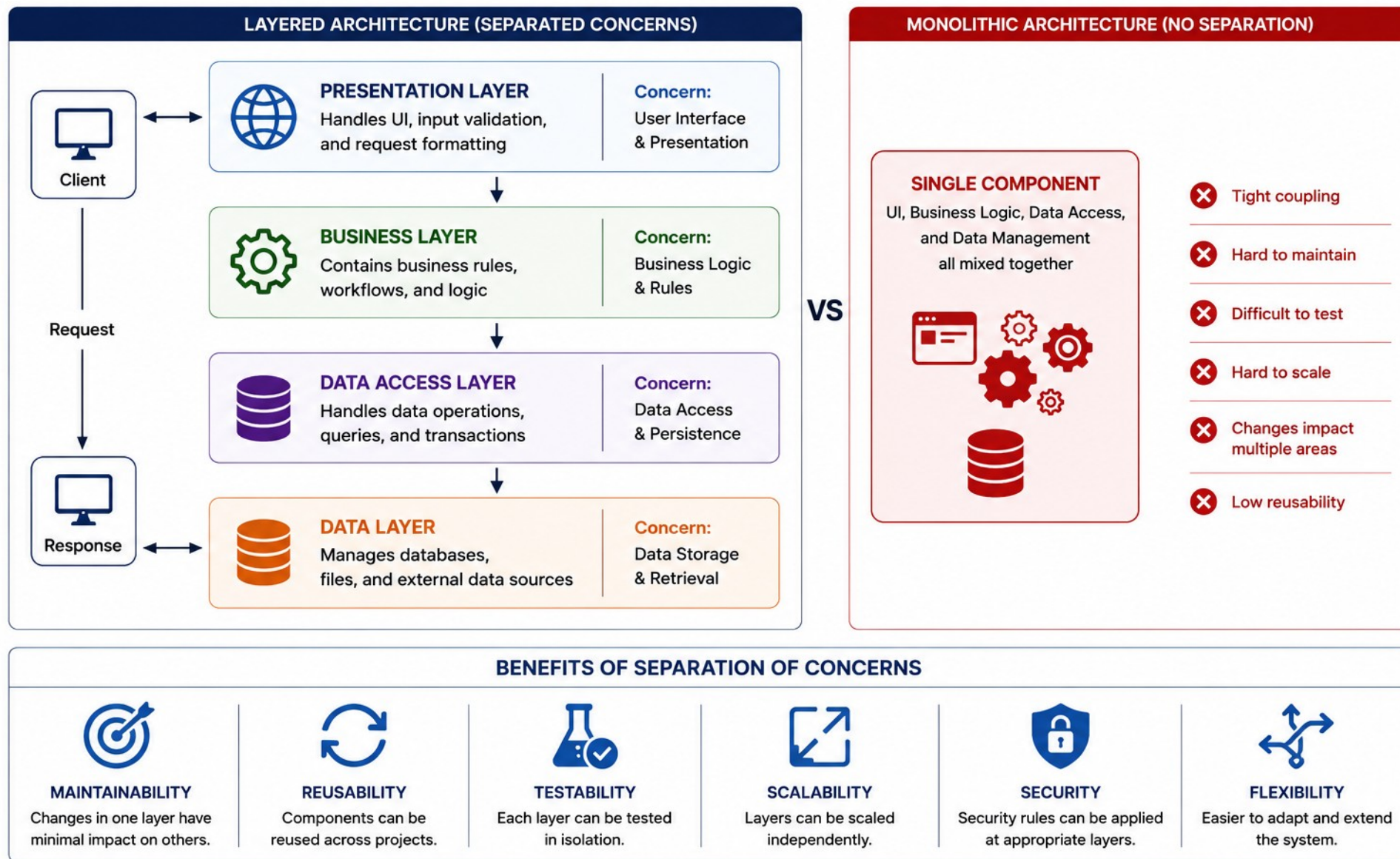
Tight coupling between web layer and data access.

Clean separation enables independent evolution and reuse.

Why it matters: better testability, readability, flexibility, and team collaboration — each layer evolves and is tested independently.

KEY TAKEAWAY Keep dependencies deliberate and directed toward stable abstractions. Presentation should not access persistence directly, and business logic should remain independent of delivery and storage details.

SEPARATION OF CONCERNS



SERVICE LAYER PATTERN

Encapsulates business logic in a dedicated layer between the controller and the repository layer.

SERVICE RESPONSIBILITIES (WHO DOES WHAT)

- **Application service** — coordinates the use case, defines the transaction boundary, calls repositories/collaborators, and translates failures.
- **Domain object or domain service** — implements business rules, protects invariants, and performs domain calculations.
- **Validator or policy** — handles reusable rule evaluation where appropriate (e.g., input shape, cross-field checks).

PLAIN JAVA EXAMPLE (LAB)

```
public final class DefaultCustomerService
    implements CustomerService {
    private final CustomerRepository repository;
    private final CustomerValidator validator;
    public DefaultCustomerService(
        CustomerRepository repository,
        CustomerValidator validator) {
        this.repository = repository;
        this.validator = validator;
    }
}
```

In Spring, the implementation may later be registered with `@Service`, and transaction boundaries may use `@Transactional`.

BENEFITS



Clean Separation



Easier Testing



Better Maintainability

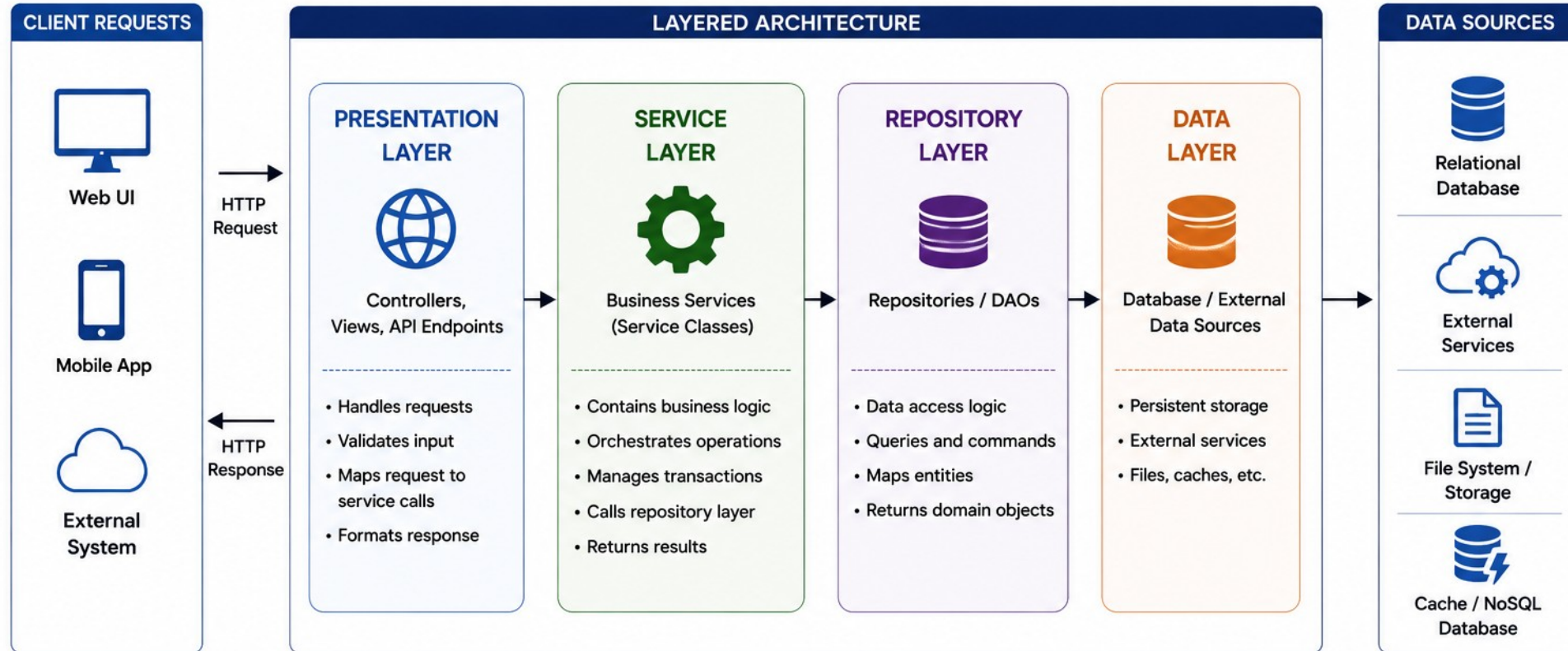


Supports Scalability

Example flow: Controller → Service (validate, apply rules) → Repository (save) → Service → Controller responds.

KEY TAKEAWAY The Service Layer Pattern keeps business logic organized, reusable, and independent of the web and data access layers.

SERVICE LAYER PATTERN



BENEFITS OF SERVICE LAYER PATTERN



SEPARATION OF CONCERNS

Keeps business logic separate from presentation and data access logic.



REUSABILITY

Business services can be reused across multiple controllers and clients.



TESTABILITY

Business logic can be tested independently from other layers.



MAINTAINABILITY

Changes in one layer do not impact other layers.



SCALABILITY

Supports scaling by isolating business logic.



TRANSACTION MANAGEMENT

Centralized transaction handling in the service layer.

REPOSITORY LAYER PATTERN

The Repository abstracts data access — the service layer works with domain objects, not SQL or persistence details.

KEY RESPONSIBILITIES

- Encapsulate data access logic (CRUD operations).
- Hide persistence details (JPA, JDBC, etc.) from the service layer.
- Provide a collection-like interface for domain objects.
- Enable easy mocking for unit tests.

SPRING DATA JPA EXAMPLE

```
public interface UserRepository
    extends JpaRepository<User, Long> {
    Optional<User> findByEmail(String email);
}
```

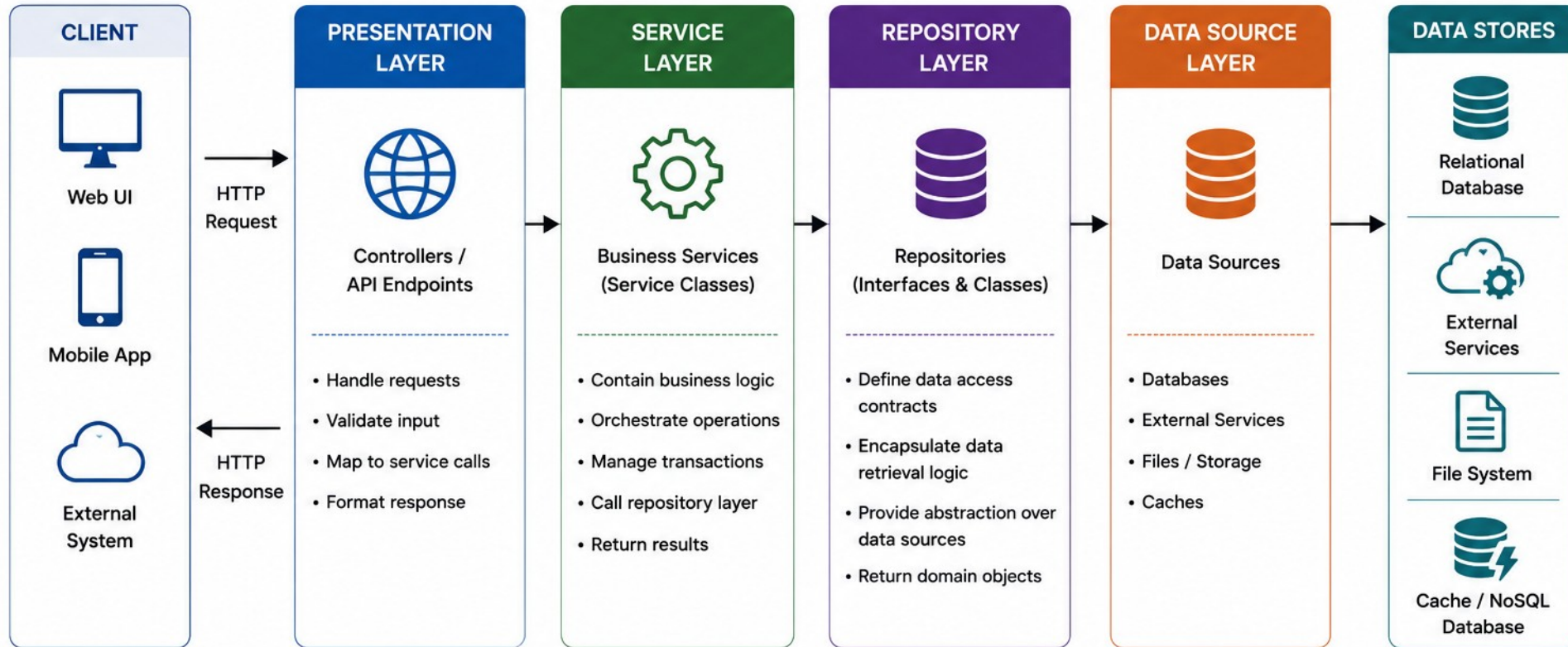
BEST PRACTICES

- Repositories encapsulate persistence operations and domain-oriented queries. They should not own orchestration, workflow decisions, or business-policy evaluation. Queries like `findActiveCustomersByRegion(...)` or `existsByEmail(...)` belong here — workflow does not.
- In the standard layered architecture used in this course, controllers call application services rather than repositories directly. Specialized read models may use separate query handlers in more advanced designs.
- Introduce repository abstractions where they protect domain boundaries, support testing, or isolate persistence choices — avoid wrappers that merely duplicate a framework interface. This module's custom `CustomerRepository` interface is justified: it separates the service from the Map-backed implementation.

Repository flow: Service calls repo method → JPA executes query → maps results to entities → returns to service.

KEY TAKEAWAY The Repository Layer isolates persistence concerns, so your service layer can evolve independently of the database.

REPOSITORY LAYER PATTERN



KEY BENEFITS



ABSTRACTION

Hides data access details from business logic.



SEPARATION OF CONCERNS

Business logic is separate from data access logic.



TESTABILITY

Repositories can be mocked for unit testing.



FLEXIBILITY

Easy to switch data sources without changing business logic.



MAINTAINABILITY

Centralized data access logic reduces code duplication.

BUSINESS RULE PLACEMENT

Business rules encode the 'why' of your application — where a rule belongs matters as much as how it's written.

RULE TYPE	APPROPRIATE LOCATION
Input shape	DTO validation
Entity invariant	Domain object
Multi-entity rule	Domain / application service
Workflow orchestration	Application service
Database integrity	Repository / database constraint
Authorization policy	Application / security policy

- **Name rules for what they enforce** — `ensureCustomerCanBeActivated()`, `calculateEligibleDiscount()`, `rejectDuplicateEmail()`, `verifyCreditLimit()`, `applyDiscount()`.
- **Separate simple & complex rules** — simple validation vs. complex domain logic.
- **Avoid duplication** — extract shared rules into reusable methods/classes.
- **Make rules testable** — pure functions where possible; easy to unit test.
- **Document business intent** — comment the why, not the how.

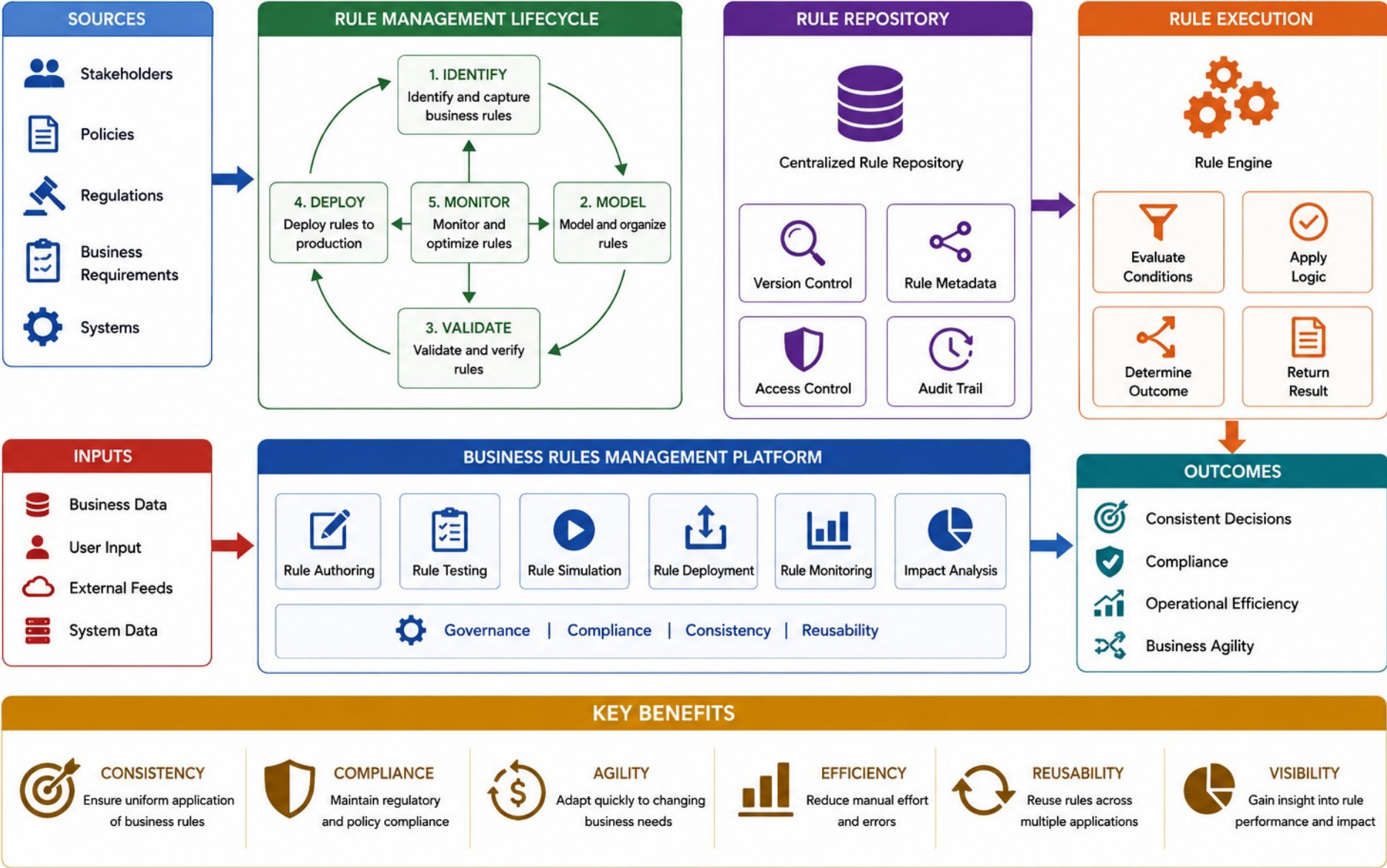
Domain-object style: `customer.activate();` — *preferable when the entity itself owns the invariant.*

Procedural service-only style: `customerService.validateStatusTransition(customer.getStatus(), ACTIVE);` `customer.setStatus(ACTIVE);`

Common types of business rules: Validation, Calculation, Authorization, Workflow, and Integrity rules.

KEY TAKEAWAY Well-managed business rules make your application easier to understand, test, and evolve as requirements change.

BUSINESS RULES MANAGEMENT



DEPENDENCY INJECTION REVIEW

DI keeps the service layer testable — dependencies stay visible, explicit, and easy to swap for mocks or fakes.

EXAMPLE: CONSTRUCTOR INJECTION

```
@Service
public class OrderService {
    private final OrderRepository repo;
    public OrderService(OrderRepository repo) {
        this.repo = repo;
    }
}
```

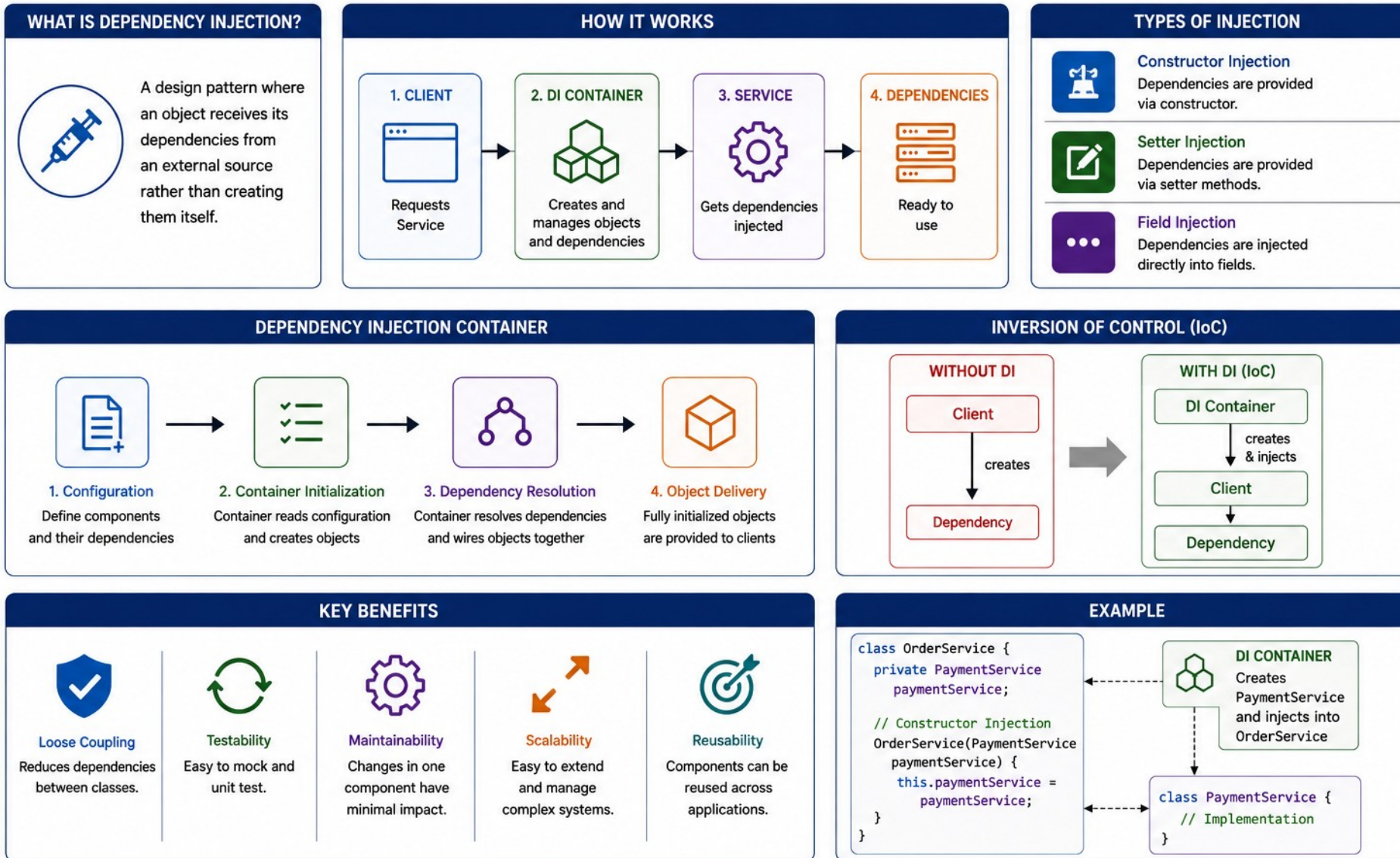
WHY IT MAKES SERVICES TESTABLE

- Dependencies are visible in the constructor signature — no hidden wiring to trace.
- Fields are final/immutable references — a wired service can't be reconfigured behind your back.
- Easy manual wiring — plain `new DefaultCustomerService(repo, validator)` works without a framework.
- Easy to mock or fake — pass a test double directly into the constructor.
- No hidden service locator — collaborators aren't pulled from a static registry mid-method.
- Avoid unnecessary optional constructor dependencies — they signal an unclear responsibility.

Use constructor injection for required collaborators. Setter injection may be appropriate for optional or reconfigurable collaborators. Avoid field injection because it hides dependencies and complicates testing.

KEY TAKEAWAY Constructor injection makes your services loosely coupled, explicit about dependencies, and easy to test in isolation.

DEPENDENCY INJECTION REVIEW



TRANSACTION BOUNDARY CONCEPT

A preview of transaction boundaries — one use case is one local unit of work. Detailed propagation and isolation rules are covered in the dedicated transaction module.

THE CORE IDEA

- One use case = one logical unit of work.
- Local database changes should succeed or fail together.
- Application services commonly define the boundary.
- Detailed propagation and isolation rules are covered in the dedicated transaction module.

EXAMPLE (SPRING) — LOCAL OPERATIONS ONLY

```
@Transactional
public void placeOrder(OrderRequest request) {
    orderRepository.save(order);
    inventoryRepository.reserveStock(request.items());
}
```

REMOTE CALLS ARE NOT PART OF THE TRANSACTION

- Remote calls require distributed-workflow patterns such as outbox, saga, compensation, or idempotent retries — not a normal local database transaction.
- Keep the transaction short and scoped to one use case; do not hold it open across a network call.

Transaction rollback behavior depends on the framework and exception configuration. In Spring, unchecked exceptions normally trigger rollback by default; checked exceptions may require explicit configuration.

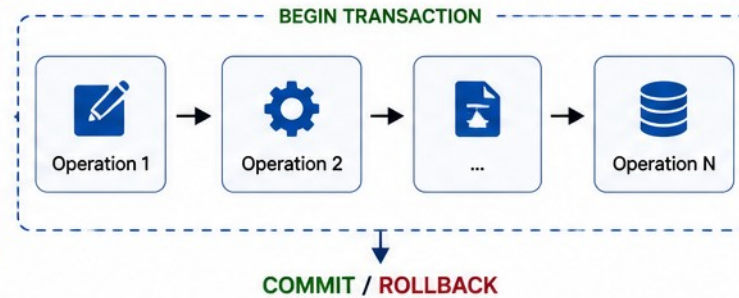
KEY TAKEAWAY A transaction boundary is a local unit of work — commonly scoped to one application-service use case — and kept free of remote calls.

TRANSACTION BOUNDARIES

WHAT IS A TRANSACTION BOUNDARY?



A transaction boundary defines the scope of work that must be completed as a single unit. Either all operations succeed (**COMMIT**) or all fail (**ROLLBACK**).



ACID PROPERTIES

- A** **Atomicity**
All operations succeed or none.
- C** **Consistency**
Database remains in a valid state.
- I** **Isolation**
Concurrent transactions do not interfere.
- D** **Durability**
Committed changes are permanent.

WHERE TO DEFINE TRANSACTION BOUNDARIES?

PRESENTATION LAYER



UI / API Controller

Should be thin.
Not the right place for transactions.

SERVICE LAYER (RECOMMENDED)



Define transaction boundaries around business use cases.

Example Use Case:
Transfer Money
(debit from A, credit to B)

REPOSITORY LAYER



Execute data operations.
Should not control transaction boundaries.

DATABASE



Enforces ACID
at the database level.

EXAMPLE: TRANSFER MONEY

- 1 BEGIN TRANSACTION
 - 2 Read balance of Account A
 - 3 Debit amount from Account A
 - 4 Credit amount to Account B
 - 5 Update transaction records
 - 6 COMMIT
- Group 1-5: All succeed → COMMIT
- Group 6: Any failure → ROLLBACK

GOOD PRACTICES



Align with Business Use Cases

One transaction per business operation.



Keep it Short

Minimize duration to reduce locks and improve performance.



Do Not Cross Boundaries Unnecessarily

Avoid transactions spanning external systems.



Handle Exceptions Properly

Rollback on failure, commit on success.



Monitor and Log

Track slow or failed transactions.

BAD EXAMPLES (AVOID)



Long-Running Transactions

Holding locks for too long impacts concurrency.



User Interaction Inside Transaction

Never wait for user input inside a transaction.



External Calls Inside Transaction

APIs, messaging, or I/O calls can cause timeouts and inconsistencies.

SERVICE COMMUNICATION — ARCHITECTURE PREVIEW

This module focuses on in-process service-layer design. Cross-service and distributed communication are previewed here only; a dedicated module covers them in depth.

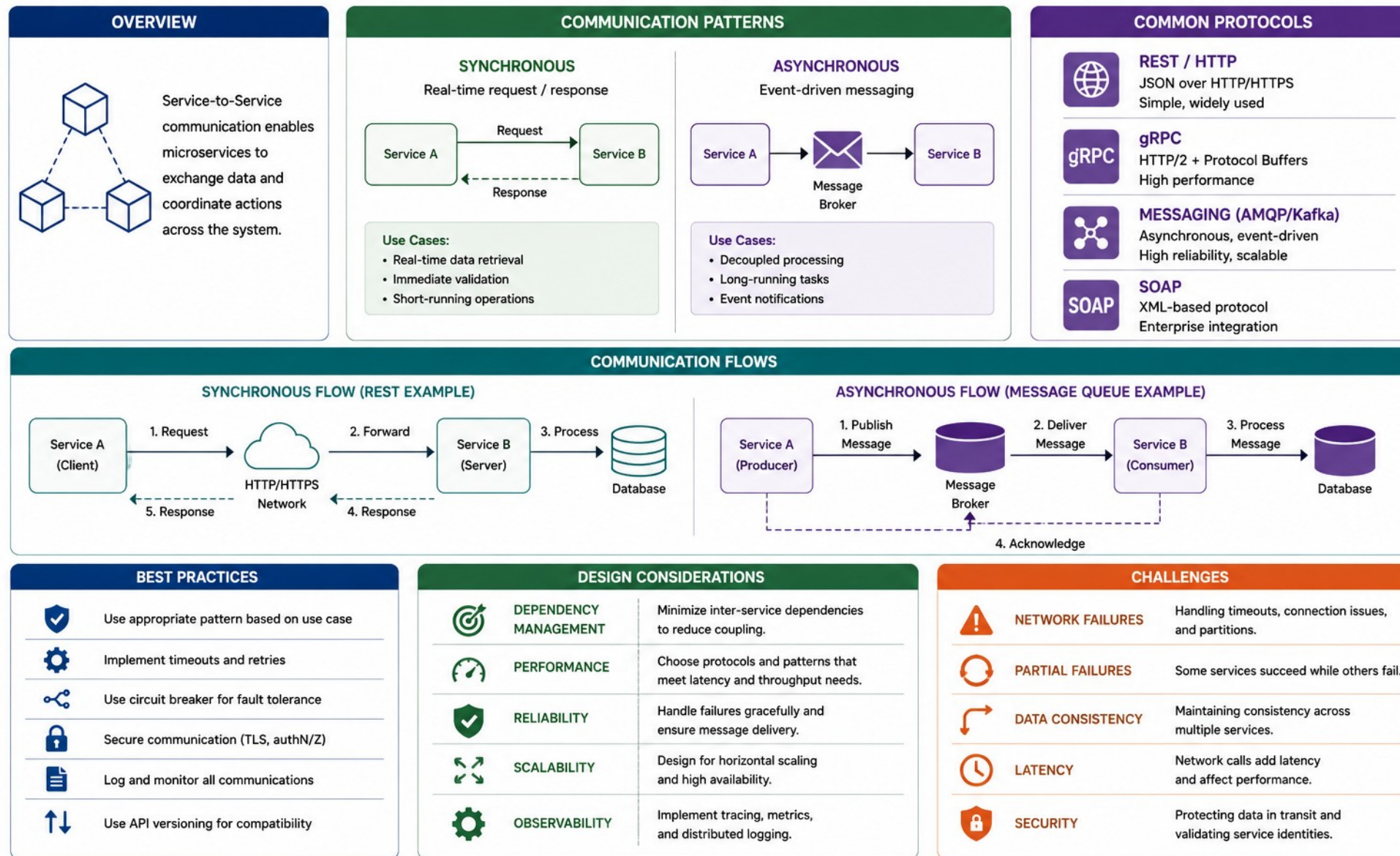
STYLE	BEST FIT	MAIN TRADE-OFF
In-process call	Same application or module	Tight runtime coupling
REST / HTTP	Simple, interoperable request-response	Temporal coupling
gRPC	Typed, high-throughput internal calls	Stronger tooling / protocol coupling
Messaging / events	Asynchronous workflows and decoupling	Eventual consistency and complexity

This module's service layer (and Lab 15) uses only in-process, direct method calls — CustomerService calls CustomerRepository directly, with no network hop.

Messaging supports asynchronous decoupling and load buffering, but requires explicit handling of retries, duplicates, ordering, and eventual consistency.

KEY TAKEAWAY Choose a communication style deliberately — most of this module's work stays in-process; cross-service styles get full treatment in the distributed-systems module.

SERVICE-TO-SERVICE COMMUNICATION



SERVICE DESIGN QUALITY ATTRIBUTES

Six quality attributes to check for any service — on top of the architecture patterns already covered in this module.



Correctness

Each use case behaves as specified, with rules enforced deliberately.



Testability

Program to interfaces so behavior can be verified in isolation.



Security

Validate inputs and enforce authorization at every service boundary.



Observability

Logging, metrics, and tracing built in.



Performance

Avoid N+1 queries; use caching where appropriate.



Reliability

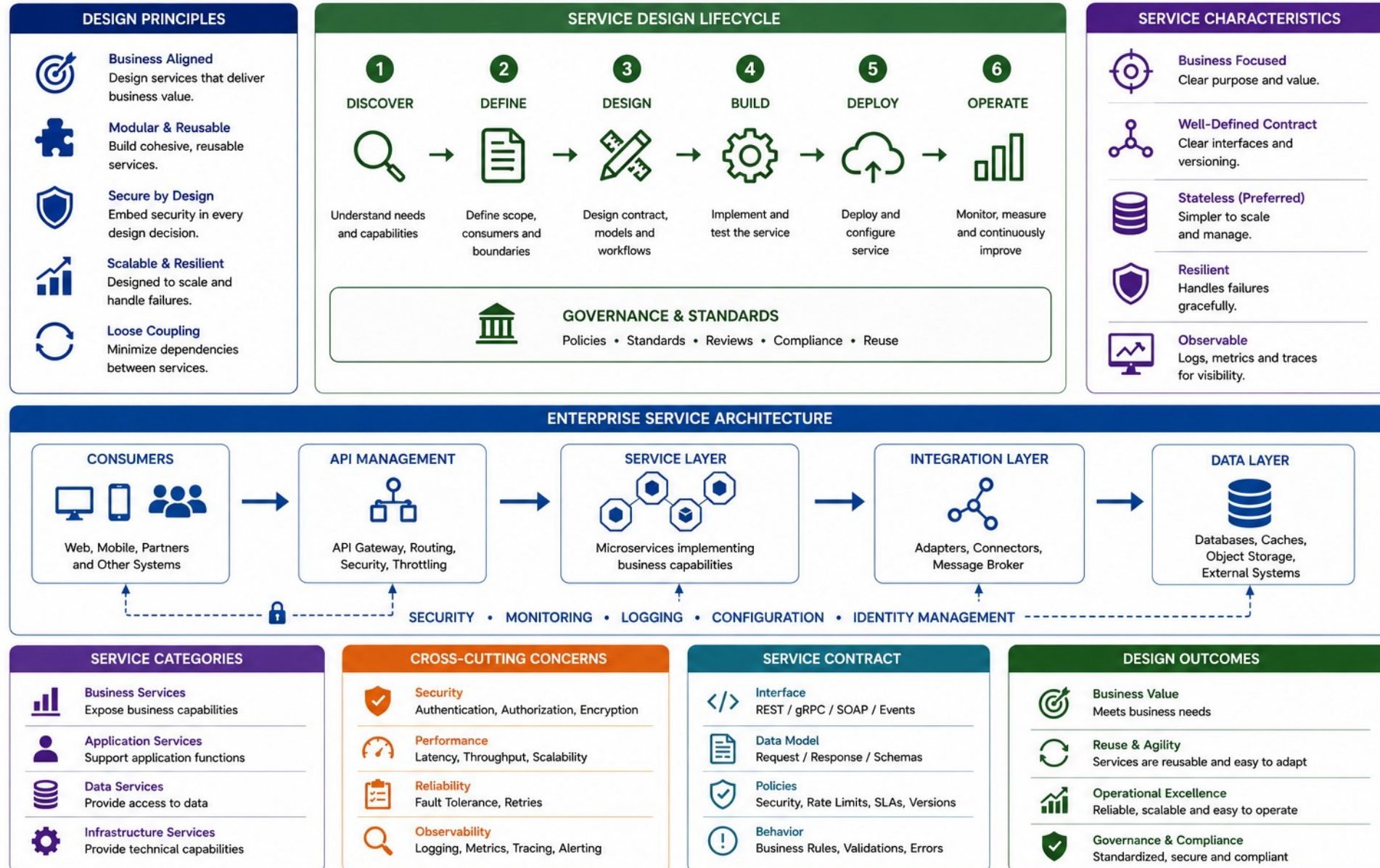
Persist durable state externally; avoid request-specific mutable state in instances.

Data ownership — In microservice architectures, each independently deployed service should generally own its data. In a modular monolith, enforce clear ownership boundaries even when infrastructure is shared.

Statelessness — Keep service instances free of request-specific mutable state where possible. Persist durable state externally so multiple instances can process requests safely.

KEY TAKEAWAY Use these attributes as a review checklist for any service you design — not a new set of architecture rules on top of what you've already learned.

ENTERPRISE SERVICE DESIGN



COMMON MISTAKES & PREFERRED DESIGN

Each mistake paired with its impact and the preferred design that avoids it.

MISTAKE	IMPACT	PREFERRED DESIGN
Business logic in controllers	Tight coupling, poor testability	Keep business logic in the service — not in controllers or repositories.
Bypassing the service layer (controllers call repositories directly)	Violates separation of concerns	Follow layered architecture: Controller → Service → Repository, always.
Anemic service layer	Thin services, duplicated logic	A purely pass-through service may be unnecessary if it adds no boundary, policy, orchestration, or abstraction. A thin application service can still be valid when domain objects contain the business behavior.
Incorrect transaction boundaries	Performance issues, inconsistent data	Scope one local unit of work per use case, defined at the application service.
Tight coupling between services	Hard to change, test, deploy independently	Use loose coupling — interfaces between services, constructor injection.
Ignoring error handling & logging	Hard to troubleshoot and monitor in production	Handle errors where meaningful recovery or translation is possible. Log once at an appropriate boundary with correlation and context; avoid duplicate stack traces across layers.
Weak or missing unit tests for business logic	Regressions ship silently; refactors become risky	Write unit tests — test business logic in isolation with mocks.

KEY TAKEAWAY Good design is about making the right things easy and the wrong things hard — use the preferred-design column as your default.

COMMON DESIGN MISTAKES

1



High Coupling

Components depend on too many others, making changes risky and difficult.

2



Low Cohesion

A component has too many responsibilities, violating the single responsibility principle.

3



Code Duplication

Duplicated logic increases maintenance effort and the chance of inconsistencies.

4



Leaky Abstractions

Internal implementation details are exposed, causing tight coupling and fragility.

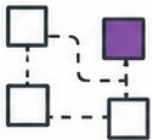
5



Ignoring SOLID Principles

Violating SOLID principles leads to unmaintainable and inflexible designs.

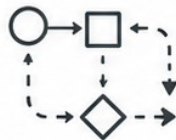
6



God Object / God Class

A single class does too much, becomes complex and hard to test.

7



Poor Layering

Mixing concerns across layers leads to spaghetti dependencies.

8



Ignoring Data Boundaries

Sharing data directly across services breaks encapsulation and causes side effects.

9



Over-Engineering

Building overly complex solutions for simple problems wastes time and resources.

10



No Extensibility (Closed Design)

Designs that don't anticipate change are hard to extend and adapt.

LAB OVERVIEW – SERVICE LAYER DESIGN

Design and implement a clean, layered service architecture for the Northstar CRM Customer Management Platform — no Spring required yet.

WHAT YOU WILL BUILD

- A CustomerService layer for the Northstar CRM Customer Management Platform.
- Add customers and change status: PROSPECT → ACTIVE, with illegal transitions rejected.
- Validate identity, email uniqueness, and status transitions before saving.
- Status-transition rules may live in a validator or in domain behavior — both are valid.

TECH STACK

ITEM	VALUE
Language	Java 21
Framework	Plain Java (manual DI)
Data Access	Repository interface (Map-backed)
Database	In-memory Map (no DB)

ARCHITECTURE FLOW



CustomerValidator (or a domain policy) is attached to CustomerService and is called before persisting changes.

Transition rule: the lab branches from Lab 14's facade, and adds failure experiments and evidence capture after testing.

KEY TAKEAWAY You will build a robust, enterprise-grade service layer that is maintainable, testable, and aligned with best practices.

LAB TASKS AND DELIVERABLES

Build the repository, service, and validator layers, then activate and reject customer status transitions.

#	TASK	DETAILS	FORMAT
1	Project Setup	Copy lab14-crm to lab15-crm; add the repository package.	n/a
2	Repository Layer	CustomerRepository interface + InMemoryCustomerRepository (private Map).	.java
3	Service Layer	CustomerService interface + DefaultCustomerService with constructor DI.	.java
4	Validator	CustomerValidator: identity, email uniqueness, status transitions.	.java
5	Status Transitions	Activate CUS-1002 PROSPECT→ACTIVE; reject ACTIVE→PROSPECT — the domain exception carries a stable error code, and the facade adds the correlation ID.	.java
6	Testing & Docs	CustomerValidatorTest + DefaultCustomerServiceTest (JUnit 5): create, duplicate email, valid/invalid transition, repo-not-called-on-failure, error-code checks; transition table in README.	Test suite

SUCCESS CRITERIA

- No Map/SQL leaks past the repository; illegal transitions rejected with a stable error code, and the facade attaches the correlation ID.
- The repository enforces email uniqueness as the final integrity guarantee — service validation only improves the error message.
- Service depends on the CustomerRepository interface (not InMemoryCustomerRepository); constructor args are non-null; returned collections are not mutable internal references; no static/global repository state.
- CustomerValidatorTest and DefaultCustomerServiceTest both pass and run independently, using a mocked or in-memory fake repository.

KEY TAKEAWAY Focus on clean design, separation of concerns, and maintainability throughout the lab.

SERVICE LAYER: DEFINITION OF DONE

A checklist to confirm your service-layer design is complete before calling a use case done.

- 1 The use case has one clear entry point.
- 2 The controller or facade contains no business workflow.
- 3 Business invariants are placed deliberately.
- 4 Dependencies are explicit and constructor-injected.
- 5 Persistence is accessed through an abstraction.
- 6 Transaction scope matches one local unit of work.
- 7 Remote calls are not assumed to be part of a local transaction.
- 8 Errors have stable codes and useful context.
- 9 Logging is not duplicated.
- 10 Unit tests cover successful and rejected workflows.
- 11 Infrastructure can change without rewriting business logic.

Use this as a pre-review checklist for any new use case in the service layer.

KEY TAKEAWAY A service layer that satisfies this checklist is maintainable, testable, and safe to extend.

KNOWLEDGE CHECK (1 OF 2)

Test your understanding of business logic and service layer design.

1. A controller validates eligibility, checks duplicate email, saves via a repository, and sends a notification. What should be refactored?

- A. Nothing — this is standard controller responsibility
- B. Move orchestration into a service; keep controllers thin**
- C. Move the repository call into the controller for speed
- D. Add more validation directly in the repository

3. Which is a sign of an anemic service layer?

- A. Rich business logic in services
- B. Services only delegate to repositories**
- C. Well-tested business rules
- D. Clear separation of concerns

2. Where are transaction boundaries most commonly defined in a layered application?

- A. In the Controller
- B. In the Repository
- C. Around application-service use cases**
- D. In the Database

4. Why prefer constructor injection?

- A. It's faster at runtime
- B. Dependencies are explicit and easy to test**
- C. It requires less code
- D. It avoids using interfaces

KEY TAKEAWAY The Service Layer owns business logic and transactions — keep it rich, testable, and well-bounded.

KNOWLEDGE CHECK (2 OF 2)

Four more questions, plus the full answer key.

5. What's a common mistake in layered design?

- A. Using interfaces between layers
- B. Controllers calling repositories directly**
- C. Writing unit tests for services
- D. Using dependency injection

7. What does the Repository layer do?

- A. Contains business rules
- B. Handles HTTP responses
- C. Encapsulates data access logic**
- D. Manages UI state

ANSWER KEY

- **1-B 2-C 3-B 4-B 5-B 6-B 7-C 8-B**

6. A service writes to the database, then calls a remote payment API inside @Transactional. What is the main problem?

- A. The code won't compile
- B. The local transaction can't atomically roll back the remote call**
- C. @Transactional doesn't work with repositories
- D. The service should use more repositories

8. Why avoid tight coupling between services?

- A. It improves performance
- B. It makes services hard to change and test**
- C. It's required by Spring Boot
- D. It reduces the number of classes

KEY TAKEAWAY Good design is about making the right things easy and the wrong things hard.

MODULE 15 COMPLETE

Business Logic and Service Layer Design

You can now design clean, testable service and repository layers with proper transaction management.